Agentic Architectures, LLM Options, and Personal Health Context Design

Research brief for the NOVA Health AI engine (Phase 1, Weeks 1-2)

Project	NOVA Health: AGI-Powered Personal Health Intelligence Companion
Supervisor	Mr. Diluka Wijesinghe
Module	ai-engine/ (AI research task)
Date	13 October 2026
Status	Draft v0.1 for team review

1. Summary and recommendations

This brief answers three questions: which agentic architecture suits NOVA Health, which LLM to use, and how the user's personal health context should be stored and kept up to date. The short answers:

1. **Architecture:** a router, one tool-using ReAct assistant, fixed workflows for scans and reports, and a rule-based safety layer. Do not start with multi-agent.
2. **LLM:** no model training. Use a pre-trained model through an API, with two tiers (a cheap model for extraction and simple chat, a mid-tier model for reasoning and reports) behind a provider-agnostic wrapper. Pick the final provider after a small bake-off test.
3. **Context:** the LLM has no memory, so the database is its memory. Store facts in structured tables, keep a compact context snapshot per user, update it by events plus a nightly job, and build each prompt from a small token budget.
4. **Privacy:** health data is sensitive. Send the model only what the task needs, use pseudonymous IDs, keep consent and audit records, and use fake data during development.

> **Terminology note:** true AGI does not exist yet. In reports, describe NOVA as an "AGI-inspired agentic architecture": a pre-trained LLM that reasons over the user's data and uses tools.

2. Agentic architectures

An AI agent is a control loop wrapped around a language model: it receives input, decides what to do, calls tools, observes the result, and repeats until it can answer. The main building blocks are the model, memory, tools, an orchestration layer, and guardrails.

2.1 Common patterns

Pattern	How it works	Strengths	Weaknesses	Use in NOVA
Single LLM call	One prompt in, one answer out.	Simple, cheap, predictable.	No tools, no live data.	Text parts of scans, summary wording.
Workflow (prompt chain)	Fixed steps in code; the LLM fills each step.	Predictable, easy to test and debug.	Rigid; cannot adapt.	Prescription scan, food scan, daily report.
ReAct (reason + act)	Model picks a tool, reads the result, repeats.	Flexible, transparent, common default.	Variable cost and latency; can loop; needs step limits.	Assistant chat that reads sleep, meds, logs.
Plan-and-execute	Make a full plan first, then run the steps.	Good for long multi-step jobs.	Brittle if conditions change mid-run.	Not needed in v1; maybe weekly insights.
Reflection	A second pass critiques the first answer.	Better quality and safety checks.	Doubles cost and latency.	Small safety checker on health advice.

Pattern	How it works	Strengths	Weaknesses	Use in NOVA
Router	A classifier picks the path or model.	Saves cost, keeps paths separate.	Needs a reliable classifier.	Route chat: emergency, data question, FAQ.
Multi-agent	Several specialist agents coordinate.	Specialization, scales to big systems.	High complexity, cost, hard to debug.	Future work only (nutrition agent, medication agent).

Sources: public agent-pattern guides (see References). Rule of thumb from these guides: start with the simplest pattern that works, and add complexity only after a specific problem appears.

2.2 Recommended architecture for NOVA

Router + one ReAct assistant + fixed workflows + rule-based safety.

```
User message / event
   |
   v
[ Router ] ---- emergency keywords or abnormal readings ---> [ Fixed rules: emergency flow ]
   |
   +---- scan or report job --------------------------------> [ Workflow: OCR/vision -> JSON ]
   |
   +---- question about my health ---> [ ReAct assistant + tools + health context ]
                                                  |
                                                  v
                                       [ Safety check: no diagnosis, disclaimer ]
                                                  |
                                                  v
                                            Reply to app (saved to DB)
```

Why this design:

- Scans and reports follow the same steps every time, so fixed workflows are cheaper and easier to test than an agent.
- Open questions ("why am I tired this week?") need flexible data lookups, so the assistant uses the ReAct loop with tools.
- Emergency detection must never depend on the LLM alone. Use fixed thresholds and rules, and use the LLM only to word the message.
- One agent is easier to debug, cheaper, and fits a 12-week student project.

2.3 Tools the assistant can call

Tool	Purpose	Type
get_profile()	Static profile: goals, allergies, preferences	Read
get_active_medications()	Current medicines and schedule	Read
get_logs(type, days)	Sleep, water, mood, meals, symptoms for a date range	Read
get_wearable_summary(date)	Steps, heart rate, sleep from the watch	Read
get_weather()	Current weather for health tips	Read
create_reminder(...)	Create a reminder	Write, needs user confirmation
propose_fact(...)	Suggest a new fact for the profile	Write, saved as pending

Tool rules: read tools by default; write tools need user confirmation; step limit of 5 tool calls per request; timeouts on every call; log every call for audit.

3. LLM options

No model training is needed. Training a medical model would need large datasets, GPUs, and clinical validation, which a 12-week project cannot support. The work is in the data, prompts, tools, and safety rules around a pre-trained model.

3.1 Options and indicative prices

Provider	Model	Input $/1M	Output $/1M	Notes
Anthropic	Claude Haiku 5.5	0.10	0.50	Cheapest Claude tier; rates rise for prompts above 100K tokens.
Anthropic	Claude Sonnet 5.5	2.00	10.00	Mid-tier; 1M context; cached reads cheaper.
Anthropic	Claude Opus 5.5	4.00	20.00	Flagship tier; likely more than needed.
OpenAI	GPT-6 Luna	0.10	0.50	Budget tier.
OpenAI	GPT-6 Sol	2.00	10.00	Mid-tier.
Google	Gemini 3.5 Flash-Lite	0.30	2.50	Budget tier.
Google	Gemini 3.8 Flash	0.75	3.75	Intro price through 31 Dec 2026, then 1.50 / 7.50.
Google	Gemini 3.1 Pro (preview)	2.00	12.00	Higher rate above 200K-token prompts.
Self-hosted	Open-weight model (e.g. via Ollama)	n/a	n/a	Data stays local, but needs a GPU, setup time, and quality is usually lower.

Prices are taken from third-party comparison pages dated September to October 2026 and change often. **Confirm every price on the provider's official pricing page before deciding.**

3.2 Selection criteria

Criterion	What to check	How to test
Cost	Price per task and per user per month	Count tokens on sample requests
Vision accuracy	Reads prescriptions and food photos	20 fake prescription and food images
Tool calling	Calls the right tool with the right arguments	10 chat prompts that need data lookups
Safety behavior	Refuses to diagnose, adds a disclaimer, escalates serious symptoms	10 risky prompts (chest pain, overdose questions)
Privacy terms	Data retention, training opt-out, region of processing	Read the provider's API data policy
Limits and access	Rate limits, free tier or student credits, SDK quality	Check docs and dashboard
Latency	Time to answer on a phone	Time each test call

3.3 Recommended strategy

1. **Two tiers.** Use a budget model (Haiku-class, Luna-class, or Flash-Lite-class) for scan extraction, routing, and simple chat. Use a mid-tier model (Sonnet-class, Sol-class, or Flash-class) for cross-domain reasoning and daily reports.
2. **Provider-agnostic wrapper.** Keep all model calls behind one function (call_llm in engine.py) so the provider can change without touching the rest of the code.
3. **Bake-off in Week 2.** Run the same 30 test cases (scan, chat, safety, report) on 2 or 3 providers. Score each 1-5, and record cost and latency. Choose with data.
4. **No flagship tier in v1.** Top-tier models cost several times more and are not needed for this project's tasks.

3.4 Rough cost estimate

Assumption: one user per day sends about 20,000 input tokens (context, chat, report) and receives about 2,000 output tokens.

Tier	Price used (in / out per 1M)	Cost per user per day	10 test users, 30 days
Budget	$0.10 / $0.50	about $0.003	about $0.90
Mid-tier	$2.00 / $10.00	about $0.06	about $18

Caching, shorter context, and the two-tier split all reduce this further. Check each provider for free tiers or student credits before spending money.

4. Personal health context: storage and updates

An LLM does not remember the user. Each request starts blank. The **personal health context** is what the system loads from the database and puts into the prompt so the model can answer as if it knows the user. So the context design and the database design are directly connected.

4.1 Context layers

Layer	What it holds	Changes	Stored as
1. Static profile	Goals, allergies, preferences, emergency contacts	Rarely; user edits	user_profile, allergies, emergency_contacts
2. Active state	Current medications, active symptoms, current goals	Often; events	medications, symptom_logs (status)
3. Raw logs	Sleep, water, mood, meals, wearable samples	Continuously	sleep_logs, water_logs, mood_logs, meal_logs, wearable_samples
4. Derived summaries	Daily and weekly summaries, averages, trends	Nightly job	daily_summaries, health_context
5. Conversation memory	Chat history and user-approved facts from chat	Per message	chat_messages, pending_facts
6. Insights log	Generated reports, alerts, advice	When generated	ai_insights

4.2 Rules for every stored fact

- **Provenance:** every fact records its source (user, scan, wearable, AI), a timestamp, and a status (confirmed or pending).
- **Trust order:** user-confirmed beats scan-extracted, which beats AI-inferred. AI-inferred facts stay pending until the user confirms them.
- **History, not overwrite:** when a fact changes (for example a medicine is stopped), mark the old row as ended instead of deleting it.
- **Numbers come from SQL, not the LLM:** averages, counts, and trends are computed with queries. The LLM only writes the explanation.

4.3 Context snapshot (what the AI sees)

The health_context row holds one compact JSON snapshot per user. It is rebuilt from the tables, so it can always be regenerated. Example with fake data:

```
{
  "user_ref": "u_8f3a",
  "profile": {"age_range": "20-29", "allergies": ["peanuts"], "goals": ["sleep 7h"]},
  "active_medications": [
    {"name": "Sample Medicine", "dose": "500 mg", "times": ["08:00", "20:00"],
     "source": "scan", "status": "confirmed"}
  ],
  "active_symptoms": [{"name": "headache", "since": "2026-10-10"}],
  "last_7_days": {"sleep_avg_h": 5.5, "water_avg_ml": 970,
                  "low_mood_days": 3, "steps_avg": 5200},
  "trends": ["sleep below goal on 6 of 7 days", "water below 1500 ml on 5 of 7 days"],
  "weather_today": {"temp_c": 31, "humidity": 80},
  "updated_at": "2026-10-13T06:00:00Z"
}
```

Note: no name, phone number, or address appears in what is sent to the model. user_ref is a pseudonymous ID.

4.4 How the context is updated

Event	Immediate action	Scheduled action
New log (water, sleep, mood)	Save row; mark snapshot as stale	Nightly: recompute daily summary and 7-day figures
Medication added or stopped	Update active medications; refresh reminders	None
Prescription scan	Save as pending; ask user to confirm; on confirm, add medication	None
Wearable sync	Store samples	Nightly: roll up into the daily summary
Chat reveals a new fact	Store as pending_fact; ask the user to confirm	None
Weather change	None	Morning: refresh weather_today
Old data piles up	None	Weekly: roll daily summaries into weekly; archive raw rows

Update cycle in order:

1 An event arrives from the app or a wearable and is written to its table.

2 Rules update the fast fields right away (active medications, active symptoms).

3 A nightly job runs SQL aggregates and rule-based trend flags, then asks the LLM to write the short narrative for the daily summary.

4 The job rebuilds the health_context snapshot and sets updated_at.

5 A weekly job compacts old summaries and archives raw rows.

4.5 Building the prompt (token budget)

Part of the prompt	Approx. tokens	When included
System prompt (role, safety rules)	300	Always
Static profile and active state	400	Always
7-day summary and trends	300	Always
Weather	50	When relevant
Last 6 chat messages	600	Chat requests
Detail on demand (tool results)	0 to 1,000	Only if the assistant asks via a tool

Total is about 1,500 to 2,500 tokens per request. Start with plain SQL queries by date and type. A vector database is not needed in v1; consider it later only for searching long chat history or health articles (RAG).

4.6 Tables to add to the ERD (for the database owner)

Table	Key columns
user_profile	user_id, user_ref, age_range, goals, created_at
allergies / emergency_contacts	user_id, item, source, status, updated_at
medications	user_id, name, dose, times, start_date, end_date, source, status
sleep_logs / water_logs / mood_logs / meal_logs / symptom_logs	user_id, value fields, logged_at, source
wearable_samples	user_id, metric, value, recorded_at, device
daily_summaries	user_id, date, stats (JSON), narrative, created_at
health_context	user_id, snapshot (JSON), updated_at, version
pending_facts	user_id, fact, origin, status, created_at, decided_at
chat_messages	user_id, role, text, created_at
ai_insights	user_id, type, content, model, created_at
consent_records / audit_log	user_id, action, detail, created_at

5. Privacy and safety

- **Data minimization:** send the model only the fields the task needs; no names, phone numbers, or addresses.
- **Security:** HTTPS, encryption at rest, per-user access control, keys in environment variables.
- **Consent and control:** record consent, let users view and delete their data, keep an audit log of AI calls.
- **Fake data:** use sample data for development, tests, screenshots, and demos.
- **Medical safety:** guidance only, no diagnosis, a disclaimer on AI screens, and a "see a doctor" path for serious symptoms. Scan results need user confirmation.

5.1 Sri Lanka data protection note

Sri Lanka's Personal Data Protection Act treats health data as a special category. Law-firm summaries state that its key provisions commence on 1 January 2027. They also describe rules on consent for special categories, transfers of personal data outside Sri Lanka (relevant when using overseas LLM APIs), and impact assessments for profiling. This is a general note, not legal advice. Confirm requirements with the supervisor or the university before collecting real user data.

6. Next steps (Week 2)

#	Task	Owner
1	Write the 30-case test set (scan, chat, safety, report) using fake data	AI engine and QA
2	Run the bake-off on 2 or 3 providers; record score, cost, latency	AI engine
3	Confirm prices and data policies on official provider pages	AI engine
4	Add the tables in 4.6 to the ERD and review the snapshot format	Database
5	Agree the tool list and step limits with the backend	Backend and AI engine
6	Write the system prompt and safety rules (v1)	AI engine and QA
7	Replace the stub in engine.py with the chosen provider's API	AI engine
8	Ask the supervisor about PDPA and real-data handling	Team lead

7. Risks

Risk	Mitigation
LLM gives unsafe or wrong advice	Strict system prompt, output checker, disclaimer, rules for emergencies
Prices or models change	Provider-agnostic wrapper; re-check prices before the final decision
Hallucinated health facts	Compute numbers with SQL; confirm scans and AI-inferred facts with the user
Agent loops or runs up cost	Step limit, timeouts, per-user daily token cap
Sensitive data leaves the system	Minimize fields, pseudonymous IDs, review provider data terms
Scope too large for 12 weeks	Build router, ReAct assistant, and daily report first; treat multi-agent and RAG as stretch goals

References

- Developers Digest: LLM API Pricing, October 2026 (developersdigest.tech)
- Cost decision guide: Gemini API vs OpenAI vs Claude, October 2026 (aifreeapi.com)
- Spheron: LLM API Pricing 2026 (spheron.network)
- Morph: LLM API Providers 2026 (morphllm.com)
- Agent design patterns: getwidget.dev, aiagentrank.io, ai-consulting.geekyants.com (chapter 38), shakudo.io
- Sri Lanka PDPA: neelakandan.lk (Data Protection in Transition), concentric.ai, dataguidance.com